

Reporte de la práctica

Buffer circular con pulsador y matriz LED en Arduino UNO R4 WiFi

1 Datos generales

Institución	Instituto Tecnológico de Mazatlán (TecNM)
Materia	Sistemas Programables
Fecha de evidencias	16 de septiembre de 2026
Placa y programa	Arduino UNO R4 WiFi y Arduino IDE; sketch Buffer_Circular1.ino
Materiales	Pulsador de cuatro patas, protoboard, dos cables de conexión y cable USB-C. Se utiliza la matriz LED integrada.
Alimentación	Arduino conectado a la computadora mediante USB.
Configuración	Pulsador en D2 y GND; INPUT_PULLUP; Monitor Serial a 9600 baudios, según el código.

2 Objetivo

Registrar las pulsaciones de un botón mediante una interrupción y almacenarlas temporalmente en un buffer circular. Procesar los eventos en orden y mostrar el conteo en el Monitor Serial, mientras la matriz LED ejecuta una animación programada con millis().

3 Conexiones del circuito

El pulsador cierra la conexión entre D2 y GND al presionarlo. La resistencia interna, activada con INPUT_PULLUP, mantiene la entrada en HIGH cuando está suelto; al pulsarlo cambia a LOW.

Elemento	Conexión	Función
Pulsador	Dos contactos que se unen al pulsar: uno a D2 y otro a GND.	Generar el aviso de una pulsación.
Entrada D2	INPUT_PULLUP e interrupción FALLING.	Detectar la transición de HIGH a LOW.
Matriz LED	Integrada en el UNO R4 WiFi.	Mostrar una línea de cuatro LED en movimiento.
Cable USB-C	Arduino y computadora.	Alimentar la placa, cargar el programa y consultar el Monitor Serial.

La resistencia del pulsador se activa desde el programa; el montaje no requiere una resistencia externa para esa entrada.

4 Procedimiento y funcionamiento del código

- 1 Se montó el pulsador en la protoboard y se conectó entre D2 y GND del Arduino. Se alimentó la placa por USB.
- 2 Se cargó el programa y se configuró la entrada con INPUT_PULLUP. La interrupción se activa con el cambio de HIGH a LOW al presionar el botón.
- 3 La función registrarEvento() consulta micros(), aplica un filtro de 40 ms y guarda el instante del evento en el buffer de 32 posiciones, si hay espacio.
- 4 En loop(), obtenerEvento() retira un dato por vuelta y se incrementa el contador. La extracción protege brevemente las variables compartidas con la interrupción.
- 5 actualizarAnimacion() desplaza una línea de cuatro LED cada 100 ms usando millis(), sin utilizar delay(). Se observaron el montaje y los mensajes del Monitor Serial.

Organización del buffer circular

Variable o condición	Función en el programa
indiceEscritura / indiceLectura	Indican dónde guardar y de dónde retirar el siguiente evento. Al llegar a 31 regresan a 0.
eventosPendientes	Distingue el estado vacío (0) del lleno (32), aunque los índices coincidan.
Buffer lleno	Se conservan los datos pendientes y se cuentan los nuevos eventos rechazados. El aviso se revisa en loop() cada segundo.

Representación del circuito

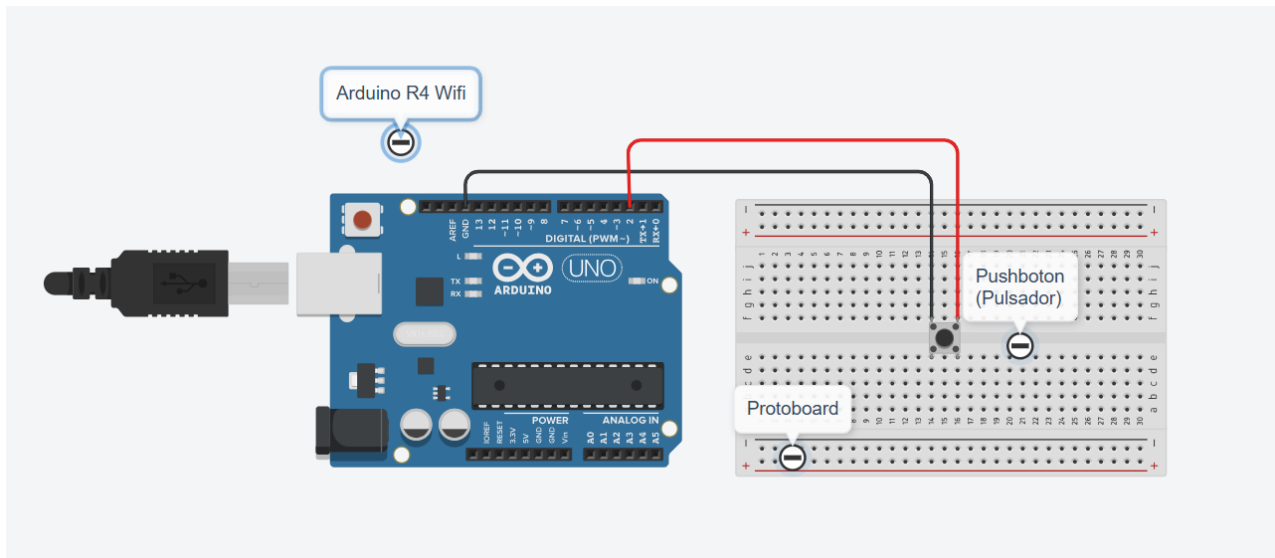


Figura 1. Esquema proporcionado del pulsador conectado al pin D2 y a GND.

El dibujo utiliza la imagen de un Arduino UNO convencional, rotulado como R4 WiFi. Representa las conexiones del botón; la placa física documentada en las fotografías es un UNO R4 WiFi.

5 Resultados

La captura del Monitor Serial muestra 30 eventos consecutivos, del #1 al #30, con marcas de tiempo crecientes. El primero aparece a los 3.953036 s del arranque y el último a los 36.792473 s. El intervalo entre ambos es de 32.839437 s.

Tabla de datos de la prueba

Evento	Tiempo registrado (µs)	Tiempo desde arranque (s)	Intervalo anterior (ms)
1	3953036	3.953036	-
2	7015336	7.015336	3062.300
3	7161613	7.161613	146.277
4	8867357	8.867357	1705.744
5	8972828	8.972828	105.471
6	10110101	10.110101	1137.273
7	10297397	10.297397	187.296
8	10379576	10.379576	82.179
9	10510252	10.510252	130.676
10	10579545	10.579545	69.293
11	10726336	10.726336	146.791
12	10793931	10.793931	67.595
13	14234029	14.234029	3440.098
14	14307140	14.307140	73.111
15	15315071	15.315071	1007.931
16	15398881	15.398881	83.810
17	15841228	15.841228	442.347
18	15958420	15.958420	117.192
19	16481498	16.481498	523.078
20	16572980	16.572980	91.482
21	17103605	17.103605	530.625
22	17214087	17.214087	110.482
23	18313371	18.313371	1099.284
24	18472284	18.472284	158.913
25	31843111	31.843111	13370.827
26	33231740	33.231740	1388.629
27	35607188	35.607188	2375.448
28	35686887	35.686887	79.699
29	36719804	36.719804	1032.917
30	36792473	36.792473	72.669

Fuente: captura del Monitor Serial proporcionada. Los tiempos en µs se transcribieron de la imagen; las conversiones a segundos y los intervalos se calcularon a partir de esos valores.

Tiempo (s) = tiempo (µs) / 1 000 000. Intervalo (ms) = diferencia entre dos tiempos consecutivos / 1 000. Para el evento #1 no hay un registro anterior disponible.

Gráficas de los resultados

Las gráficas se elaboraron con los 30 tiempos de la tabla. La primera muestra el avance del conteo; la segunda, la separación temporal entre registros. No representan el número de espacios ocupados dentro del buffer.

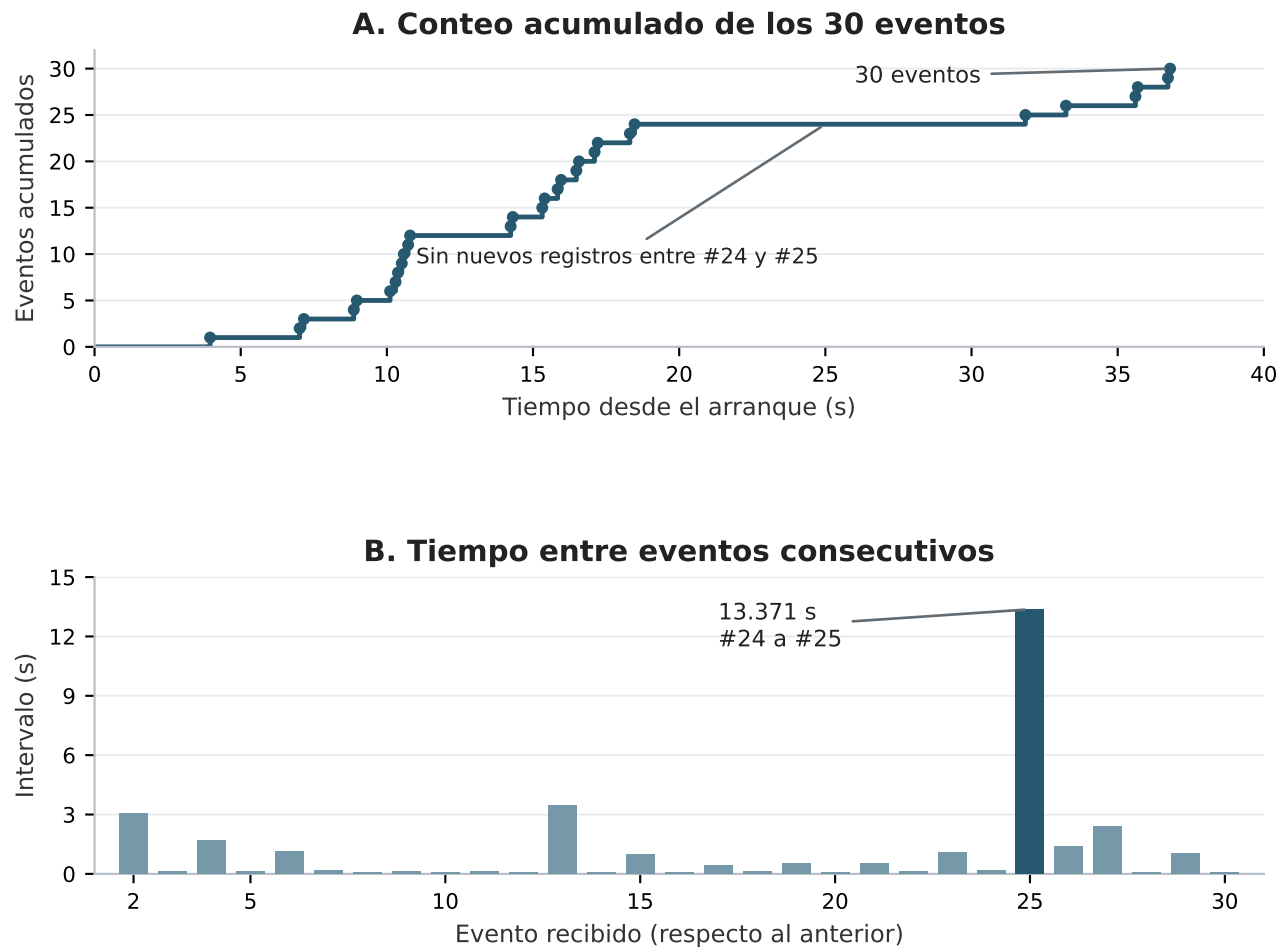


Figura 2. Conteo e intervalos calculados exclusivamente a partir de la captura del Monitor Serial.

Indicador calculado	Resultado
Eventos visibles	30
Lapso entre el primer y el último evento	32.839437 s
Menor intervalo entre eventos	67.595 ms, entre #11 y #12
Mayor intervalo entre eventos	13.370827 s, entre #24 y #25
Mediana de los 29 intervalos	158.913 ms

La sección horizontal de la gráfica A corresponde a un periodo sin nuevos eventos registrados. Por sí sola, esa pausa no demuestra una falla: la captura no indica si el pulsador se accionó durante ese intervalo.

6 Salida del Monitor Serial

La evidencia muestra la placa Arduino UNO R4 WiFi seleccionada, el puerto COM3 y el archivo Buffer_Circular1.ino. Se leen los mensajes de inicio y una secuencia numerada de 30 eventos.

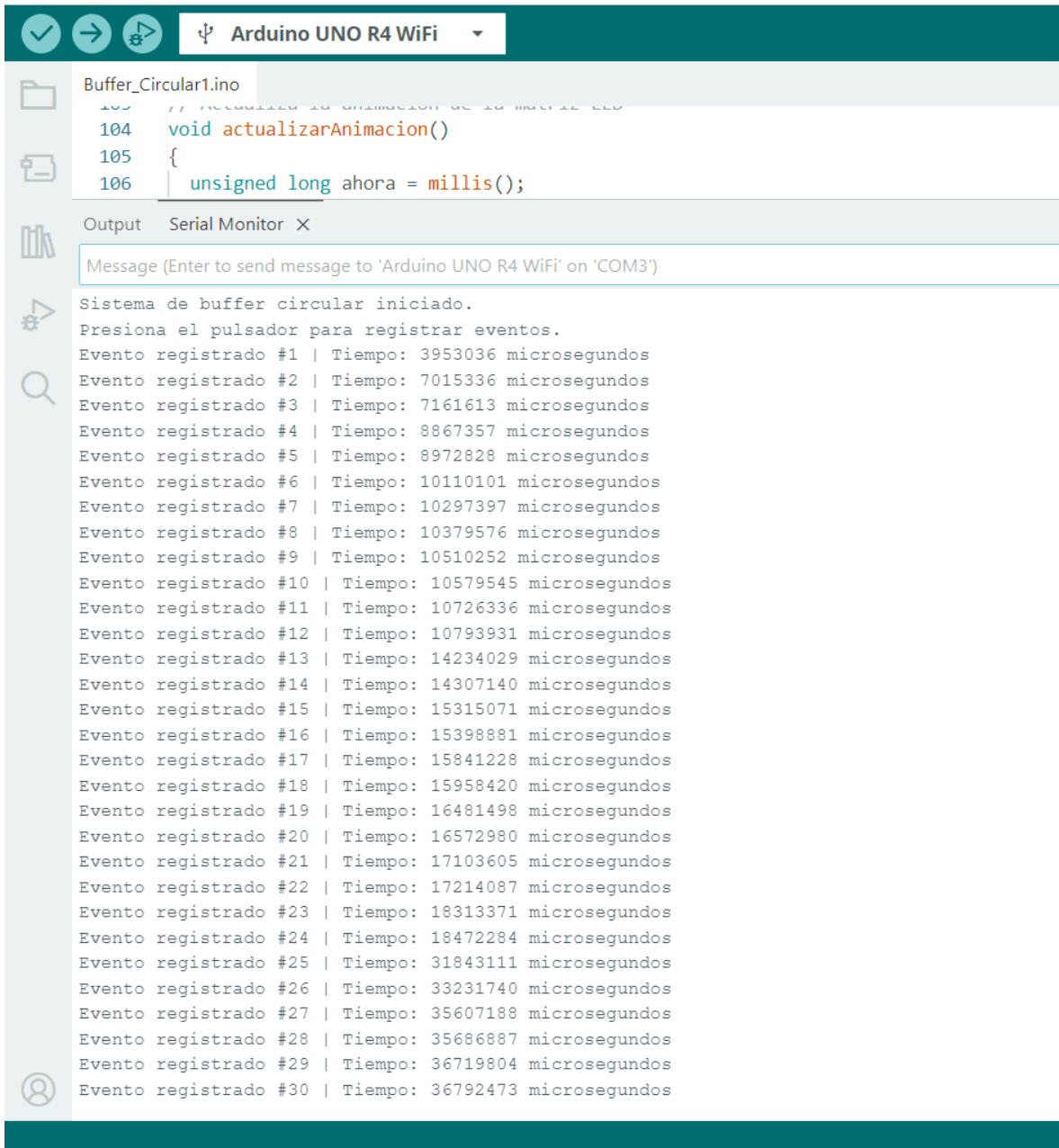


Figura 3. Captura original del Monitor Serial con los eventos #1 a #30 y sus tiempos en microsegundos.

Dato observado	Resultado
Inicio del sistema	Aparece el mensaje de buffer circular iniciado y la indicación de presionar el pulsador.
Secuencia impresa	Numeración consecutiva del #1 al #30 y tiempos crecientes.
Aviso de buffer lleno	No aparece en la porción visible del registro.

7 Observaciones sobre el comportamiento del sistema

Registro de eventos. El Monitor Serial documenta la lectura y presentación de 30 registros en orden. La numeración se genera al procesarlos; sin un conteo independiente de las pulsaciones físicas, esta captura no permite asegurar que todas se hayan detectado.

Antirrebote. El código ignora cambios separados por menos de 40 ms respecto al último evento aceptado. El menor intervalo mostrado es de 67.595 ms, compatible con ese filtro. No se registró el número de rebotes ignorados, por lo que no se mide su eficacia directamente.

Respuesta de la matriz LED. Las dos fotografías muestran la matriz encendida con un patrón de línea. El programa cambia su posición cada 100 ms. Las imágenes fijas documentan el encendido, pero no permiten medir la velocidad ni comprobar la continuidad completa de la animación.

Trabajo entre la interrupción y `loop()`. La interrupción conserva el instante de cada evento y sale; `loop()` retira datos, actualiza el conteo y atiende la animación. Las operaciones de impresión se realizan fuera de la interrupción, según el código desarrollado.

Capacidad del buffer. Las 32 posiciones corresponden a eventos pendientes de procesar, no al máximo total de pulsaciones durante una prueba. Los espacios se liberan y reutilizan. Si llegan eventos con el buffer lleno, el programa incrementa un contador de rechazos y conserva los registros anteriores.

Alcance de la prueba. Se muestran 30 eventos, una cantidad menor que las 32 posiciones del arreglo. Esta evidencia no documenta el retorno de los índices al inicio ni una condición de saturación. Ambas situaciones están contempladas en el código y requieren una prueba específica para observarlas.

Estado documentado de las funciones

Función	Evidencia disponible
Montaje físico	Pulsador, protoboard, dos conexiones y placa alimentada por USB.
Procesamiento de eventos	30 registros visibles y tiempos crecientes en el Monitor Serial.
Matriz LED	Patrón de línea encendido en las dos fotografías.
Vuelta completa del buffer	Lógica incluida en el código; no demostrada por los 30 eventos visibles.
Saturación y rechazos	Manejo incluido en el código; sin mensaje de rechazo en la captura.

Como comprobación adicional, se pueden realizar más de 32 pulsaciones válidas y comparar el conteo físico con el registro. La saturación debe evaluarse por separado, acumulando eventos pendientes de lectura.

8 Evidencias del montaje físico

Fotografías proporcionadas del circuito con el pulsador y la matriz integrada encendida. Se mantienen las imágenes completas, sin incorporar fotografías externas.

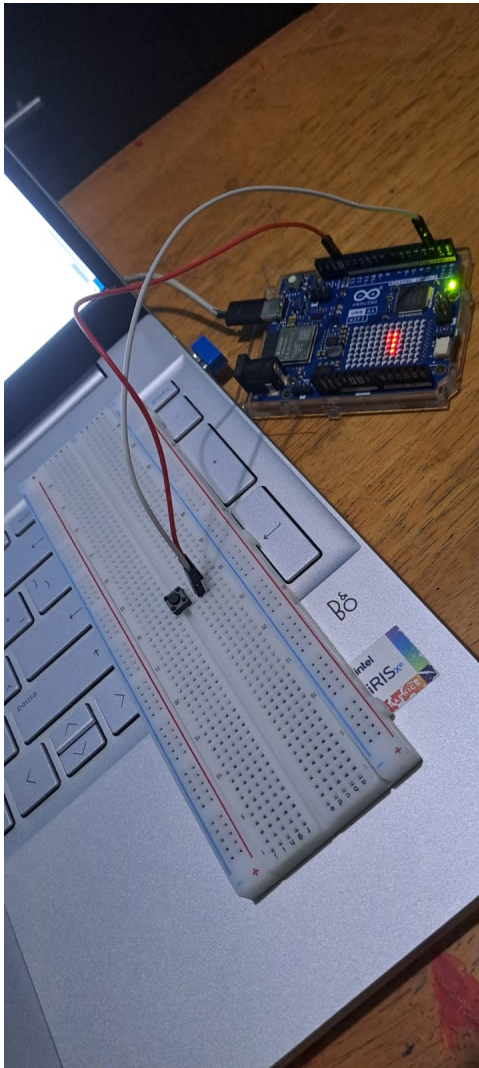


Figura 4. Vista general de la placa, el pulsador y la protoboard.

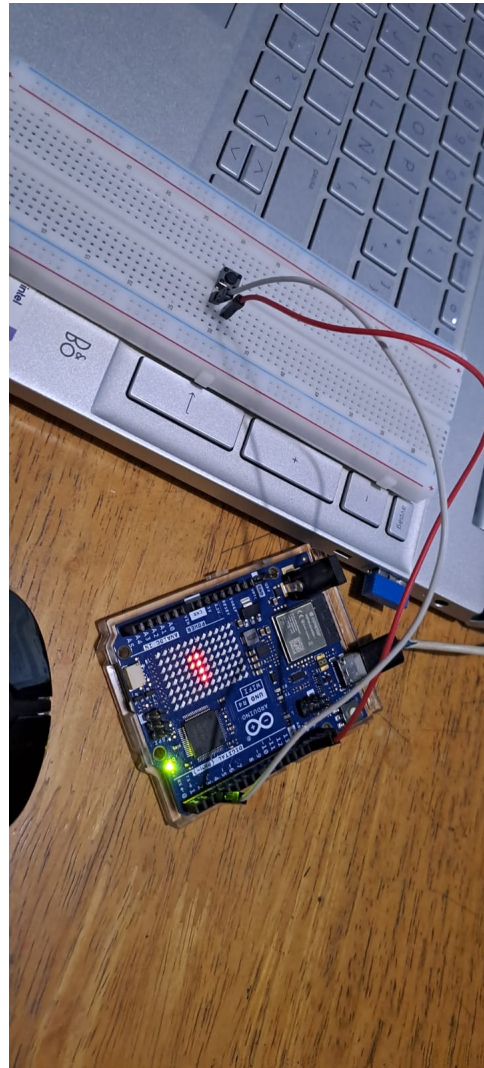


Figura 5. Segunda vista del montaje y del patrón de la matriz LED.

9 Conclusión

La práctica permitió implementar el registro de eventos con un buffer circular y documentar 30 mensajes del Monitor Serial, junto con el encendido de la matriz LED. La tabla y las gráficas muestran cuándo se registró cada evento y cuánto tiempo transcurrió entre ellos.

La interrupción guarda los avisos y `loop()` los procesa después, mientras atiende la animación. Este principio puede utilizarse para contar piezas detectadas por un sensor en una banda transportadora. Para completar la validación del buffer queda por documentar su vuelta completa y la respuesta cuando se llena.

Base del reporte: PDF de la consigna, código desarrollado en la práctica y las cuatro imágenes proporcionadas. El reporte de control por voz se utilizó únicamente como referencia de formato.